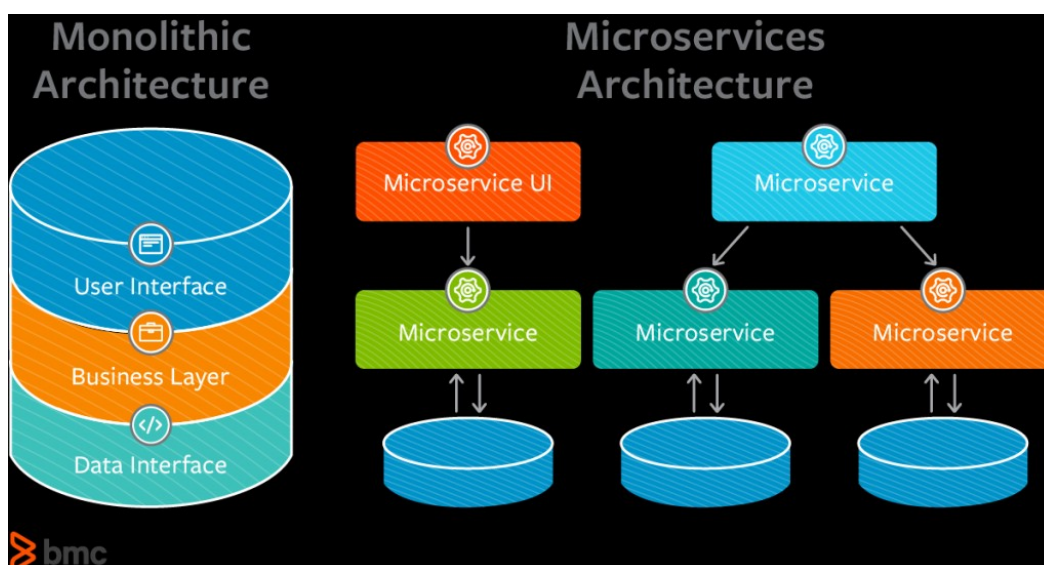


15. MICROSERVICIOS

La arquitectura basada en Microservicios es un concepto que está muy de moda y que ha sido implementado por muchos gigantes tecnológicos como Amazon, eBay, Netflix Twitter, Paypal entre otros.

Veremos la comparación de los microservicios con los sistemas monolíticos, sus ventajas, desventajas y cuando debemos implementar la arquitectura de Microservicios en PHP.

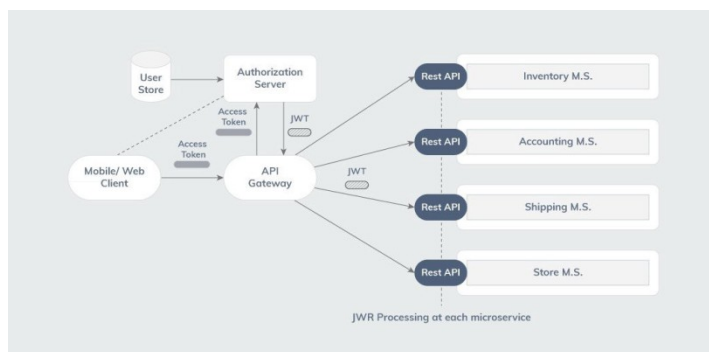
Hasta hace unos años, las grandes y complejas aplicaciones se han desarrollado siguiendo lo que se conoce como sistemas monolíticos (mono-proceso) donde el mantenimiento, escalabilidad y evolución, se estaba convirtiendo en algo traumático.



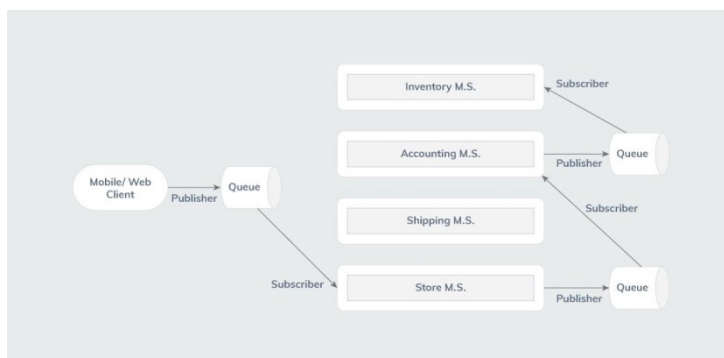
Es aquí en donde aparece la arquitectura basada en microservicios, su principio se basa en identificar y separar funcionalidad del sistema en micro proyectos o servicios autónomos que trabajan de forma independiente. Son pequeñas aplicaciones que a menudo se crean mediante una tecnología de servicio web.

Es decir, cada micro servicio corresponde por así decirlo a un módulo cuya funcionalidad es específica y se ejecutará de forma paralela en distintos servidores y comunicándose entre ellos por diferentes vías disponibles normalmente una API de recursos HTTP: Http Request, Colas de Mensajes...

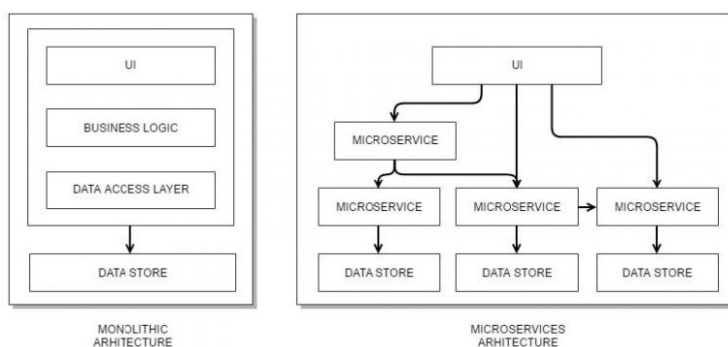
Cada microservicio es responsable de brindar solución a un área de negocio concreta, abstrayendo al resto del sistema de los detalles concretos de la implementación, favoreciendo su independencia, el mantenimiento y la evolución de cada uno de ellos.



Al gestionar cada micro servicio como un pequeño sistema, nos da la posibilidad de ser desplegados sin perjudicar a los demás, favoreciendo así la escalabilidad.



En definitiva, la idea es coger una gran aplicación monolítica y dividirla en varias aplicaciones más pequeñas, es decir, servicios, pero hacerlo de una manera que sirva a un objetivo comercial.



El estilo de microservicios se propone resolver algunos desafíos clave del desarrollo moderno de aplicaciones, algunos de los cuales incluyen lo siguiente:

- Tamaño del equipo de desarrollo: un microservicio único que puede desarrollar un equipo relativamente pequeño
- Diversidad de habilidades de desarrollo: diferentes servicios se pueden escribir en diferentes lenguajes de programación
- Cambio / actualización: fragmentos de código más pequeños son más fáciles de cambiar o actualizar
- Integración e implementación: fragmentos de código más pequeños son más fáciles de implementar
- Más fácil para los recién llegados: fragmentos de código más pequeños son más fáciles de alcanzar
- Escalabilidad: no todo se escala por igual; fragmentos de código más pequeños se pueden escalar más fácilmente
- Manejo de fallos: un único servicio defectuoso no desactiva una aplicación completa

Al mismo tiempo, inducen varios desafíos nuevos, algunos de los cuales incluyen los siguientes:

- Comunicación de servicio: este es un esfuerzo adicional en torno a la comunicación de servicio
- Pruebas y monitoreo: son algo más desafiantes que con las aplicaciones de monolitos
- Latencia de red: cada microservicio introduce un poco más de latencia de red

Ventajas

1. Potencia la diversidad tecnológica: es decir, podemos utilizar distintos stacks tecnológicos para cada microservicio, incluso, distintos lenguajes de programación.
2. Permite despliegues independientes.
3. La capacidad de testing de cada microservicio es mayor.
4. Permite que los desarrollos sean independientes y puedan avanzar en paralelo.
5. La escalabilidad es más eficiente y continua.
6. Facilita el mantenimiento a lo largo del tiempo.

Desventajas

Las desventajas se deben en gran parte a la introducción del concepto de sistemas distribuidos, algunas de estas son:

1. Aumentará la complejidad en la gestión de la configuración y despliegue porque cada microservicio va a requerir su propia configuración de build, testing, Git
2. Aumenta la complejidad para mantener la transaccionalidad de cada operación.
3. El rendimiento se puede ver afectado debido a saturaciones de la red o a procesos de (de)serialización.
4. Monitorizar el sistema será más complejo.
5. Será más difícil gestionar los errores correctamente.

La decisión de si utilizar o no este tipo de diseño a la hora de desarrollar nuestro software se fundamenta básicamente en el nivel de complejidad que va a alcanzar. Si el nivel de complejidad del software es bajo, no es aconsejable dedicar esfuerzos en implementar esta arquitectura.

A partir de cierto nivel de complejidad del sistema, las ventajas que ofrecen los microservicios harán viable el esfuerzo necesario que la productividad del equipo sea mayor.

La arquitectura de microservicios vino a resolver importantes retos y problemas relacionados a la complejidad de los sistemas actuales. Una buena implementación de los mismos garantiza que sistemas complejos puedan ser gestionados y administrados de forma eficiente.